Daric System Control Application Notes

Revision	0.2
Status	Draft
Function	CROSSBAR, Inc.
Path and Filename	Daric_system_control-AN.doc
File Type	Microsoft Word

Revision History

Rev	Status *	Date	Author	Reason for change/description
0.0	D	5/29/2025	Philip Xie	Initial draft
0.1	D	9/12/2025	Philip Xie	Updated Life Cycle system control, SKU and Debug options
0.2	D	5/12/2026	Philip Xie	Added applications

* Statuses include: D: Draft, P: Pending review/frozen, U: Update needed, A: Approved

Table of Contents

1. SYSTEM CONTROL 5
1.1 POWER CONTROL 5
1.2 RESET CONTROL 6
1.3 CLOCK CONTROL 7
1.3.1 CLOCK SCHEME 7
1.3.2 CLOCK SELECTION 8
1.3.3 STACKING FRACTIONAL CLOCK DIVIDER 9
1.3.4 CLK FREQ METER 11
1.3.5 PLL 11
1.3.6 CLK CIPHER 12
1.4 LOW POWER DESIGN 13
1.4.1 CM7 SLEEP MODE 14
1.4.2 SLEEP MODE OPTIONS 14
1.4.3 FD LP OPTION 15
1.4.4 IP LP OPTION --> IP FLOW 15
1.4.5 DEEP SLEEP OPTION 16
1.5 IP CONTROL 16
1.5.1 IP SLEEP CONTROL 17
1.5.2 PMU CONTROL 17
1.5.3 RAM CONTROL 18
1.6 AO 19
1.6.1 CLOCK AND RESET 19
1.6.2 PDMODE 19
1.6.3 POWER CONTROL 20
2. APPLICATIONS 21
2.1 CLOCK, POWER SETTINGS 21
2.1.1 BASIC PRIMITIVES 21
2.1.2 CLOCK INITIALIZATION 22
2.1.3 CLOCK FREQUENCY CONTROL AND POWER CONTROL 24
2.2 LOW POWER APPLICATIONS 26
3. DESIGN REVIEW INFORMATION 28

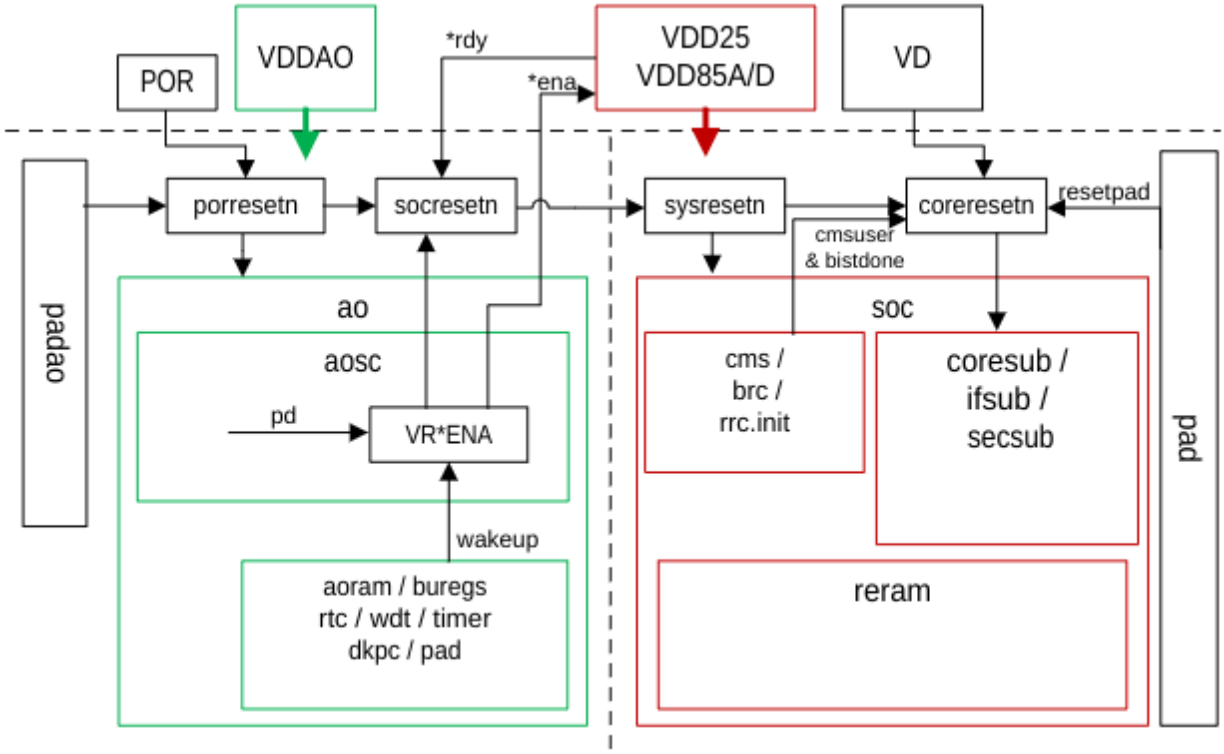
No table of figures entries found.

Table of Tables

Table 12-1 Actions Items 28

1. System control

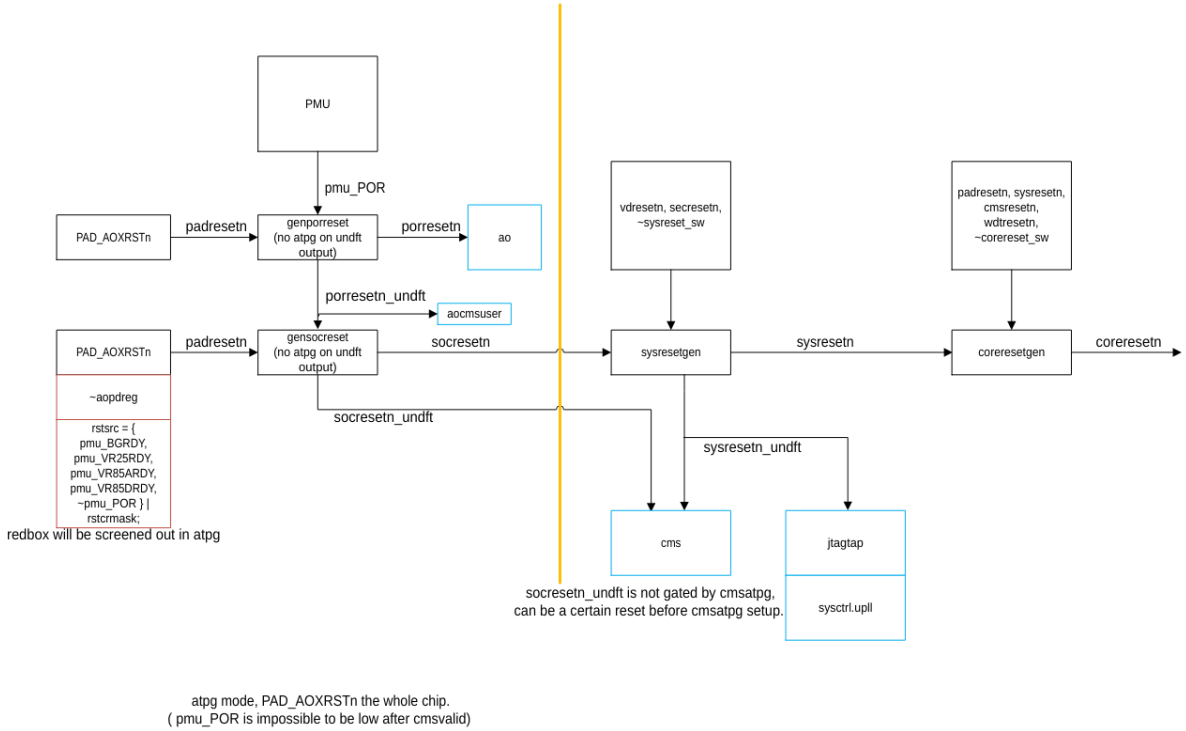
Power and reset domain



1.1 Power control

- The main power domains are VDDAO, VDD25, VDD85A/D.
- The VDDAO is always enabled.
- The VDD25/85A/D are enabled by default to power up.
- Powerdown mode, the VDD25/85A/D will be turned off together. The whole system will only have ao domain powered.
- With the pd wakeup triggered, the VDD25/85A/D will power on again.
- VDD85D can be switched between 0.85V and 0.95V
- VDD85D / VDDAO can be switched to external power supply

1.2 Reset control

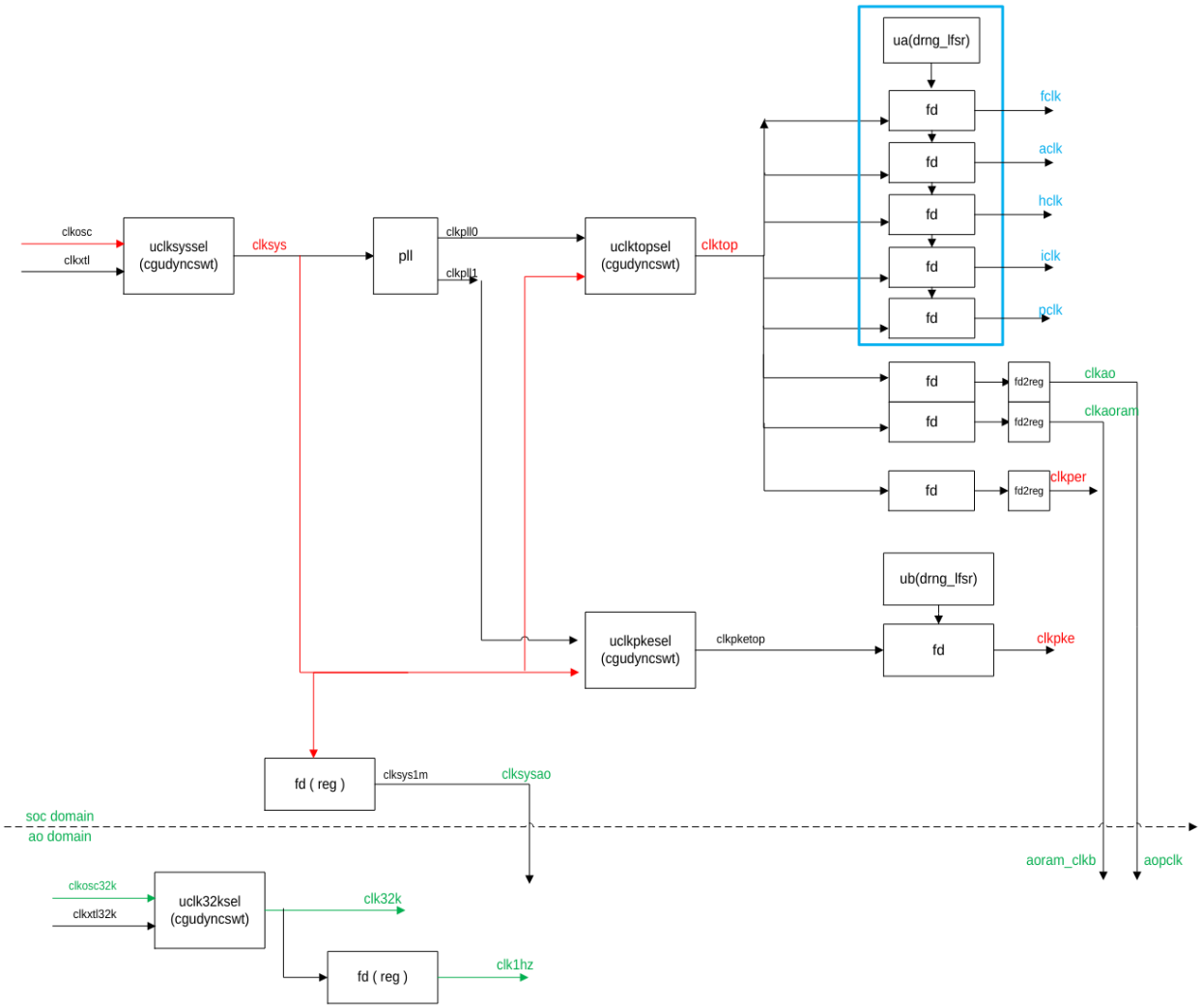


*DFT info is not the target of this document.

reset	target	source	description
porresetn	ao domain	pmu_POR	VDD33 powerup
		ao_padresetn	Pad: AOXRSTn
socresetn	soc domain	porresetn	upper-level reset
		pmu_*RDY	all VDD powersupply ready (with sw mask)
		aopdreg	powerdown mode
sysresetn	soc	socresetn	upper-level reset
		vdresetn	VD warning (with sw mask)
		secresetn	sec warning (with sw mask)
		sysreset_sw	software triggered sysreset
coreresetn	all subsys for usermode	sysresetn	upper-level reset
		padresetn	Pad: XRSTn
		cmsresetn	(cmscode == cms_pkg::CMS_USER) & brdone;
		wdtresetn	watchdog timeout
		corereset_sw	software triggered sysreset

1.3 Clock control

1.3.1 Clock scheme



Clk source:

name	frequence	accuracy	power	workmode	source
clkosc	32MHz	10%	soc	soc pon/user/dft	osc32M
clkxtl	48MHz	100 ppm	soc	soc user/dft	xtal pad
clkpll0	up to 800MHz	--	soc	soc user/dft	pll0
clkpll1	up to 300MHz	--	soc	soc user/dft	pll1
clkosc32k	32kHz	10%	ao	ao all	osc32k
clkxtl32k	32kHz	100 ppm	ao	ao all	xtal32k pad

Clk domain:

name	frequence	drv->load	func
clksys	32/48MHz	soc	sysctrl/cms/brc/rrc.init
f/a/h/i/pclk	--	soc	coresub/ifsub/secsub
clkper	100MHz	soc	ifsub
clkpke	300MHz/200MHz	soc	pke(coresub.sce.pke)
clkao	<32MHz	soc->ao	ao.pclk
clkaoram	<32MHz	soc->ao	aoram
clksysao	1MHz~1.5MHz	soc->ao	aosc(ao sysctrl)

clk32k	32.768kHz	ao	ao timer clock
clk1hz	1Hz	ao	ao timer clock

- Clksys is the most basic clock for all the system functions in soc domain:
  - All reset control
  - Chip mode selection
  - Initial bist read (IFR) and trimming
  - Usermode default clk
  - Usermode LP IP flow control
  - Can be turned off in pdmode (optional)
- Main function clock: fclk, aclk, hclk, iclk, pclk
  - usermode coresystem
  - Stacking Fractional Clock Divider: M/256
  - Fclk up to 800MHz(final signoff 750MHz) @0.9V, 500MHz @0.8V
  - All clks are synchronized
- Clkpke, the pke clk can be controlled individually and asynchronously.
- Clkper, the base clock for all the peripherals.
- Ao clock: clkaoram/clksysao/clkao/clk32k/clk1hz
  - Clkaoram, the clkaoram reading/writing clock, this clock is decided by the application, also the driving capability of VRAO.
  - Clkaoram/clksysao/clkao are invalid in the pd mode.
- Clksys/clkao are initially using internal osc32M/osc32K, till the software change the source to xtal48M/xtal32K. In this way, the chip is possible to work in the application of non-xtal components, like smartcard.

1.3.2 Clock selection

Clock source selection, internal oscillator or external Xtal.

1. Clksys
   a. Select between osc 32MHz and xtal pad 48MHz
   b. This is recommended if the application has the 48MHz xtal
   c. Use freq meter to confirm the xtal clock frequency before switching
   d. Change those config related with clksys frequency: e.g, **Cgufscr**

2. Clktop
   a. Select between clksys/clkpll0
   b. Clkpll need to be set and turned on before switching.
   c. The change also need change the clktop group fd settings

3. Clkpke
    a. In most of the application, we should use clkpll1

4. Clk32k
    a. Same with clksys, use the xtal32k if the xtal32k is valid.

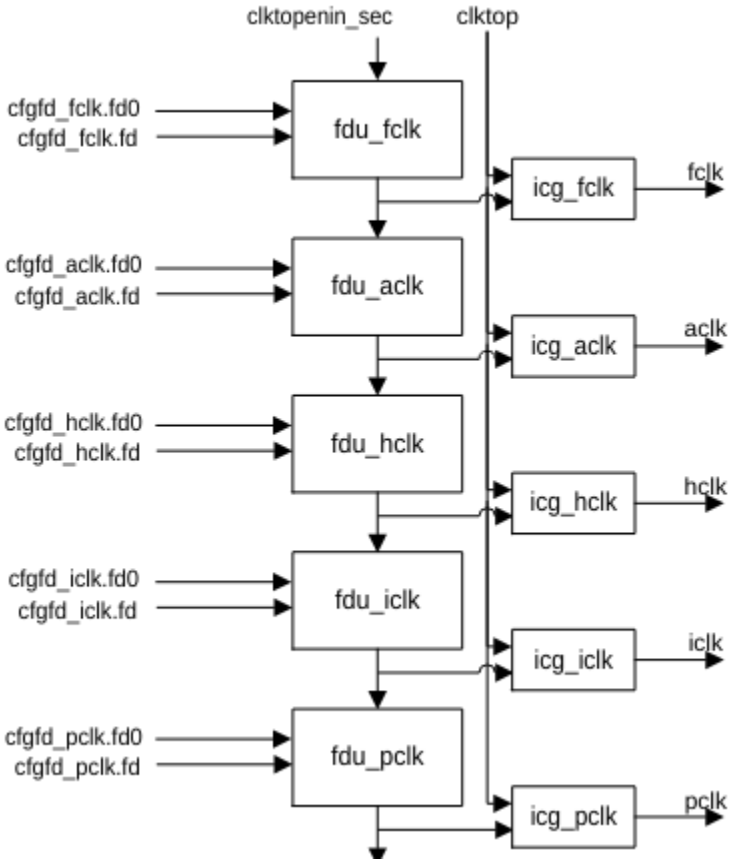
1.3.3 Stacking Fractional Clock Divider

Clock list

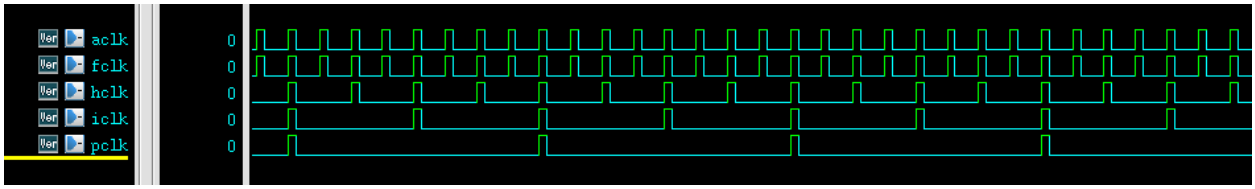
name	Target* (0.9V/0.8V)	modules	limit*	fd (max)*
fclk	800MHz/500MHz	cm7sys(cache/tcm), biodma	0	ff
aclk	400MHz/250MHz	vexsys, axi, sram0/1, qfc	1	7f
hclk	200MHz/125MHz	sce(~pke), dma, evc	3	3f
iclk	100MHz/75MHz	ifsub: udma, udc, ifram	7	1f
pclk	50MHz/37.5MHz	all apbs	f	0f

*the limit and fd are the SFR for the clock, the value is based on:
Pll0 = 800MHz @0.9V (*750MHz)
Pll0 = 500MHz @0.8V
* the target 800MHz finally signoff at 750MHz, since the violation are all about tcm. So it's still possible to use 800MHz:
-- if not using TCM (cache only), or not using CM7 (vex 400MHz+biodma 800MHz)
-- if cm7 running at 400MHz with limit = 1

1.3.3.1 Design



- The FDUs are stacked to ensure that the child clock aligns with the parent clock.
- In this design, the frequency of the child clock can never exceed that of the parent clock.
- When the child clock reaches its designated frequency division point (FD), but the parent clock has not yet issued an enable pulse, the child clock will hold and wait until it is allowed to proceed.
- The top clktopenin_sec is the random clock cipher (ua).



Our Fractional Clock Divider is a digital circuit that produces an average output frequency equal to a non-integer fraction of the input clock. It cannot divide the clock in real time by a fractional number , but instead toggles the output in a pattern that, over time, achieves the desired average frequency.
For example, fd=3/8, is actually fd=96(/256), is mixed with 128/256 (½) and 64/256(1/4), the real clock will be 2 cycles clk and 4 cycles clock mixed.
In this case the clock the minimal cycle is actually 2(8/4), which is less than the fraction 2.66(8/3).

Note since it's stacking, the child clock will possibly be slower than the number in the cfg.

1.3.3.2 Other usage of fdu

Clkao, clkaoram, clkper, clkpke are also using same fdu.

1.3.4 Clk freq meter

There are 8 freq meters in the clock system:

- fclk, to check the clock after clkcipher ua.
- Clkpke, to check the pke clock after clkcipher ub.
- Clkao
- Clkaoram
- Internal osc32M
- Internal XTAL
- Pll0
- Pll1

Check cgufsvld flag to ensure the clock is active.

1.3.5 Pll

Enable the PLL first before switching the clktop to it.
Some useful PLL from the calculator (in progmodel.xls)

pll cfg					valid range		check
	Fref	48	MHz		10	500	yes
fbdiv	N[11:0]	200	C8		12	4095	yes
prediv	M[4:0]	4	4		1	31	yes
	Fvco	2400			1000	3000	yes
	Fpd	12			10	100	yes
	Q00	2			0	7	yes
	Q10	0			0	7	yes
	Q01	3			0	7	yes
	Q11	1			0	7	yes
	clk0	800	MHz	1.25	ns		
	clk1	300	MHz	3.333333	ns		

pll cfg					valid range		check
	Fref	48	MHz		10	500	yes
fbdiv	N[11:0]	125	7D		12	4095	yes
prediv	M[4:0]	3	3		1	31	yes
	Fvco	2000			1000	3000	yes
	Fpd	16			10	100	yes
	Q00	3			0	7	yes
	Q10	0			0	7	yes
	Q01	4			0	7	yes
	Q11	1			0	7	yes
	clk0	500	MHz	2	ns		
	clk1	200	MHz	5	ns		

pll cfg					valid range		check
	Fref	48	MHz		10	500	yes
fbdiv	N[11:0]	175	AF		12	4095	yes
prediv	M[4:0]	4	4		1	31	yes
	Fvco	2100			1000	3000	yes
	Fpd	12			10	100	yes
	Q00	2			0	7	yes
	Q10	0			0	7	yes
	Q01	6			0	7	yes
	Q11	0			0	7	yes
	clk0	700	MHz	1.428571	ns		
	clk1	300	MHz	3.333333	ns		

pll cfg					valid range		check
	Fref	48	MHz		10	500	yes
fbdiv	N[11:0]	150	96		12	4095	yes
prediv	M[4:0]	4	4		1	31	yes
	Fvco	1800			1000	3000	yes
	Fpd	12			10	100	yes
	Q00	2			0	7	yes
	Q10	0			0	7	yes
	Q01	5			0	7	yes
	Q11	0			0	7	yes
	clk0	600	MHz	1.666667	ns		
	clk1	300	MHz	3.333333	ns		

pll cfg					valid range		check
	Fref	48	MHz		10	500	yes
fbdiv	N[11:0]	100	64		12	4095	yes
prediv	M[4:0]	4	4		1	31	yes
	Fvco	1200			1000	3000	yes
	Fpd	12			10	100	yes
	Q00	2			0	7	yes
	Q10	0			0	7	yes
	Q01	5			0	7	yes
	Q11	0			0	7	yes
	clk0	400	MHz	2.5	ns		
	clk1	200	MHz	5	ns		

pll cfg					valid range		check
	Fref	48	MHz		10	500	yes
fbdiv	N[11:0]	150	96		12	4095	yes
prediv	M[4:0]	4	4		1	31	yes
	Fvco	1800			1000	3000	yes
	Fpd	12			10	100	yes
	Q00	3			0	7	yes
	Q10	0			0	7	yes
	Q01	2			0	7	yes
	Q11	2			0	7	yes
	clk0	450	MHz	2.222222	ns		
	clk1	200	MHz	5	ns		

To use calculator, fill in the decimal number in red cells, if all the sanity check pass yes(green cells) then the clk0,clk1 will be the PLL outputs.

1.3.6 Clk cipher

In the clock diagram, UA and UB represent clock scrambling units. Based on software configurations, these units randomly omit a certain number of cycles from the continuous clock signal, thereby rendering the clock timing indeterminate. Ultimately, this indeterminacy makes it significantly more difficult to execute behavioral timing analyses and attacks.

- UA scrambles fclk; through stacked FD, this effect propagates to fclk, aclk, hclk, iclk, and pclk.
- UB scrambles clkpke, thereby making the operational behavior of PKE difficult to pinpoint accurately.

UA/UB both are programmable. The programmable items are:

		UA	UB	
enable	CR	cgusec[4]	cgusec[12]	clk cipher enable, default is '0
level	CR	cgusec[3:0]	cgusec[11:8]	clk cipher level setup, default is '0
reseed number	CR	cgu_seed[31:0]		the seed data to reseed
reseed action	AR	write 0x5a		set the reseed

- The clk cipher level determines the rate of the clock be gated.
  - 0x0 is full speed (no gating)
  - Each step will gate 1/16 more cycles off, but this is not accurate number
  - 0xf is the lowest speed, less then 1/16, but not stop.
- To reseed the random sequence, set the reseed number first (using TRNG is recommended), then set reseed AR to reseed the random sequence.
- There's no software timing request on these registers:
  - The enable can be set anytime and will be valid immediately .
  - The level can be set anytime and will be valid immediately.
  - The reseed can be set anytime while enabled.

Clock cipher perform effectively in terms of cycle-level timing protection, typically operating within the microsecond range or below.
However, for long-duration runtime timing analysis—spanning the milliseconds range or more—relying solely on random gating is insufficient; software-based countermeasures must be employed in conjunction:

- Randomized software delays;
- Randomized interval updates to the clock cipher gating level;

And other such techniques to provide comprehensive protection.

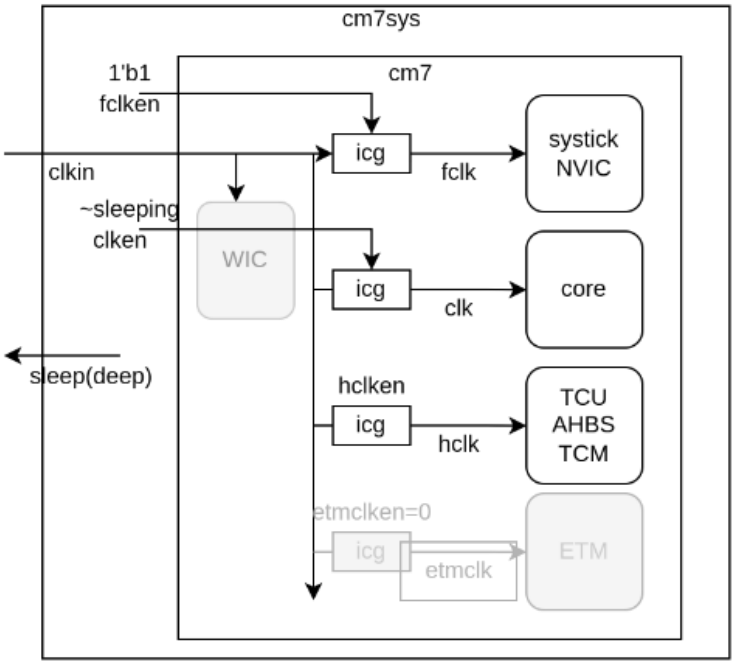
1.4 Low power design

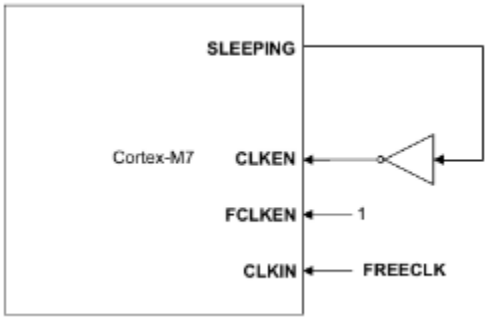
The system has two fundamental low power mode:

mode	options	enter	wakeup	Soc pwr	clktop	clksys	ip	pmu	feature
sleep mode	none	wfi/wfe	irq/ev	on	no chg	no chg	no chg	no chg	Cpu idle
	fd	wfi/wfe	irq/ev	on	lpcfg	no chg	no chg	no chg	fast wkup

	fd, ip	wfi/wfe	irq/ev	on	lpcfg	no chg	iplpcfg	pmulpcfg	wkup in us
	fd, ip, deep	wfi/wfe	async/aoev	on	no	no	iplpcfg	pmulpcfg	wkup in ms
pd mode	pmu opt	sw	aoev	off	poff	poff	poff	pmpdcfg	bootup

1.4.1 CM7 sleep mode





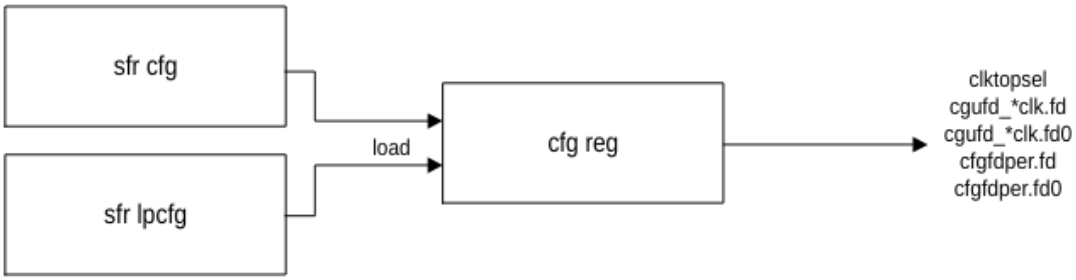
- Cm7's sleeping is enabled by the instruction WFI/WFE, the core will be clockless until the irq/ev coming.
- The optional CM7.SCR.deepsleep cfg, will send the sleep signal to the system, to trigger our sysctrl to enable the sleep mode.

1.4.2 Sleep mode options

cgulp	0x0004	0x00000000	CR	yes	cgu low power control register	*[0]:fdlpen, low power fd setting enable, only valid not deep pd [1]:iplpen, low power disable ip [2]:deepen,*

1.4.3 FD LP option

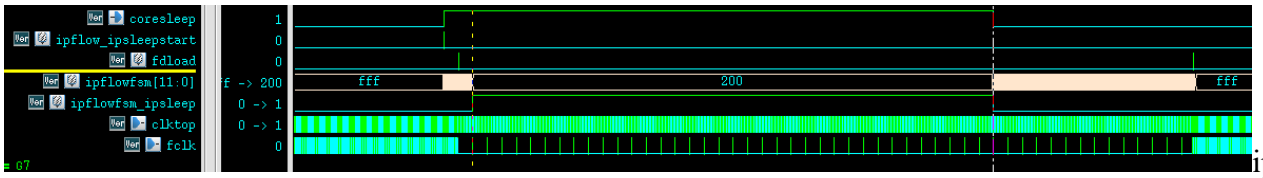
name	lpcfg	normal cfg
clktopsel	clktopsel-lp(cguselⅠ[1])	clktopsel(cguselⅠ.[0])
cgufd_*clk.fd	cgufd_*clk[15:8]	cgufd_*clk[7:0]
cgufd_*clk.fd0	cgufd_*clk[31:24]	cgufd_*clk[23:16]
cfgfdper.fd	cfgfdper[15:8]	cfgfdper[7:0]
cfgfdper.fd0	cfgfdper[31:24]	cfgfdper[23:16]



The cfg reg will be loaded together, to avoid the transition control of configs.
The lpcfg should match the rule between clock source and fd.
The load pulse will be triggered in these conditions:

load action	case	sfr cfg sel
cfgset	sfr.cguset(x002c)	sfr cfg
cfgsetinit	init after sysresetn	sfr cfg
cfgsetslp	sleep enter	sfr lpcfg
cfgsetwkup	sleep wkup	sfr cfg
cfgsetdpslp	lpflow fsm enter	sfr lpcfg
cfgsetdpwkup	lpflow fsm wkup	sfr cfg

1.4.4 IP LP option --> ip flow



ip flow is a fsm to control the timing for the ip related changing.

1. when core sleep enabled, the ipflow will be started.
2. lpflow fsm will start to count.
3. The counter will stop at the number x200, the ipsleep signal will be enabled.
4. All the related ip can use ipflow_ipsleep signal to control the workmode.
5. When the irq/ev wakeup the core. The counter will move on till getting back to 0xfff.

6. During the fsm running, the fdload also triggered to the clk system to load the cfg.

With ipflow fsm, the ip on/off/trimming will have safe guard timing to warm up.

1.4.5 Deep sleep option

With IP LP option, the core system can run slow waiting for the irq/ev, but there will still be clock running (clksys, clktop, clkpke...).

In deep sleep mode, all the internal clock will be turned off, even the xtal can be sleeping also. The system can only be wakeup by async source (async IO signal, ao event).

- async IO signal: wkupvlds_async, check iox progmodel
- ao event: aowkupvld: wkupvld_async, dkpcintr, wdtintr, tmrintr, rtcintr, wdtreset, aopad[0], aopad[1]

Since this mode will turned off xtal/PLL, the wakeup time will be longer.

1.5 IP control

		por	sfr	Trimming(ipflow)	set sfr ar(ipflow)	ipt	atpg
osc	enable	1	ipcen[0]				1
	triming	IV_OSC32MTRM		IFR	sysctrl.ipc_osc	jtag	jtag
pll	enable	0	ipcen[1]			jtag	jtag
	cfg	--	sfr_ipcpll*			jtag	jtag
rng	enable	'0	trng.cr_src			1	0
	triming	7'b0100110	trng.cr_src			ipt_rngcfg	7'b0100110
vd	enable	'1	sensorc.vden				
	triming	VD09_CFG = '0	sensorc.vden				
ts	enable	'0					
	cfg						
ld	enable	no clk					
	~~triming~~						
glue							
mesh							

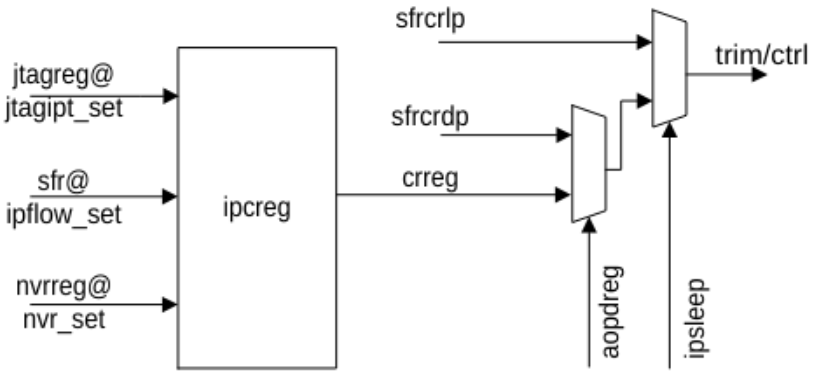
** osc/pll enable also have hardware guard to avoid all clock are off.

1.5.1 lp sleep control

The ip sleep during the ipsleep (from ipflow)

ipc_lpen	0x0098	0x00000001	CR	yes	ipc_lpen	***[0]: osc [1]: pll [2]: rram [3]: pmu [4]: pmu_IBIASENA & xtalsleep(dpsleep)***

1.5.2 PMU control



- ipflow_set, when software need to change the the value, set the sfr, and set ipflow(set sysctrl.lpcar)

- nvr_set, initial triming from IFR
- jtagipt_set, setting PMU in DFT mode
- ipsleep, the sleep mode with IP LP option, will select sfrcrlp
- aopdreg, ao domain pd mode, will select sfrcrdp

name	ipsleep	aopdreg	cmsatpg	jtagipt_set	ipflow_set	socnvr_set
pmu_ctrl	sfrpmucrlp	sfrpmucrpd	pmucrreg	socpmucr	sfrpmucr	
pmu_trm	sfrpmutrmlp	sfrpmutrmlp	pmutrmreg	socpmutrm	sfrpmutrm	socpmutrm
pmu_dft				socpmudft	sfrpmudft	
osc_ctrl	sfrosccrlp	sfrosccrpd		socosccr	sfrosccr	
osc_trm	sfrosctrmlp	sfrosctrmlp		socosctrm	sfrosctrm	socosctrm

name	signals	IV
pmu_ctrl	pmu_VDDAO_CURRENT_CFG pmu_VR25ENA pmu_VR85AENA pmu_VR85DENA pmu_IOUTENA pmu_POCENA pmu_VR85A95ENA pmu_VR85D95ENA	IV_PMUCR
pmu_trm	pmu_TRM_CUR pmu_TRM_CTAT pmu_TRM_PTAT pmu_TRM_DP60_VDD25 pmu_TRM_DP60_VDD85A pmu_TRM_DP60_VDD85D pmu_VDDAO_VOLTAGE_CFG	IV_PMUTRM
pmu_dft	pmu_TEST_SEL pmu_TEST_EN	IV_PMUDFT

1.5.3 RAM control

1.5.3.1 SRAM macro's trimming :

- Can be trimming IFR during BRC
- Can be changed by software in usermode
- Can be access by JTAG in testmode

Some macro will need to set different trimmings for different voltage 0.8/0.9V.

Sample code:

1.5.3.2 SRAM ctrl

- For 800MHz(or 750MHz) frequency fclk, the TCM will need to set fastmode.
  Sfr: sramcfg.itcm/dtcm.fastmode settings.
- For sram0 400MHz aclk, we will need to add one more waitcyc

  Sfr: sram0.waitcyc

Sample code:

1.6 AO

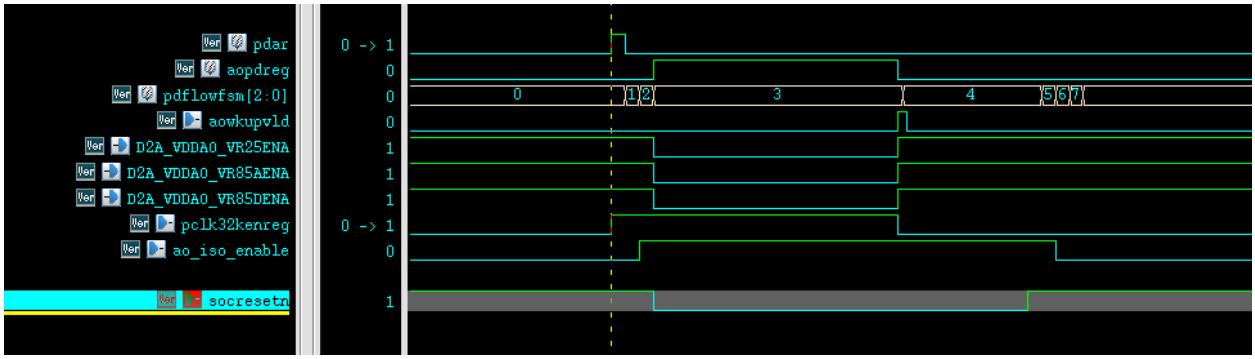
1.6.1 Clock and reset

1.6.2 Pdmode

- Enter power down mode by write SFR sfrpmupdar = 0x5a
- PMU will be switching to the value of SFR sfrpmucrpd, sfrpmutrmlp
- Set the VR25/85A/85D power off will turn the soc power off
- For the power off mode, set cr_aocr.pclkicg / pdisoen enable

cr_aocr	h0	0x00000006	apb_cr	[2]	pclkicg	pclk icg enable for pd
				[1]	pdisoen	iso enable for pd
				[0]	clk32kselreg	=0 use osc32k, =1 use xtal 32k

- ao event: aowkupvld: wkupvld_async, dkpcintr, wdtintr, tmrintr, rtcintr, wdtreset, aopad[0], aopad[1]
- If the selected 32k is not valid, the timer will not work, the soc can only wakeup by the aopad[0], aopad[1], dkpcintr_async



1.6.3 Power control

- VDD85D voltage adjustment
  - Between 0.85V ~ 0.95V
  - The digital timing will be different, need to re-config the PLL/FD
  - Use ipflow AR to set
  - Modify the SRAM trimming / SRAM0 waitcyc / TCM fastmode
- VDD85D external supply
  - When the load current exceeds the capacity of VR
  - Use ipflow AR to set
- VDDAO external supply
  - For extreme low power goal, use external power for better regulation efficiency.
- VDDAO voltage adjustment
  - For extreme low power goal, use lower VDDAO to drop the pdmode current
  - Use the minimal voltage to keep AORAM data retention and timer working correctly

2.Applications

2.1 Clock, power settings

In the work mode, clock and power can be controlled for different performance and power consumption for different purposes.

2.1.1 Basic primitives

To turn off all the sub gate of modules:

```
HWRITE   40040060 00
HWRITE   40040064 00
HWRITE   40040068 00
HWRITE   4004006c 00
```

To check clock frequency:

HWRITE cgufscr 0x30 # set the fs reference clock 48MHz

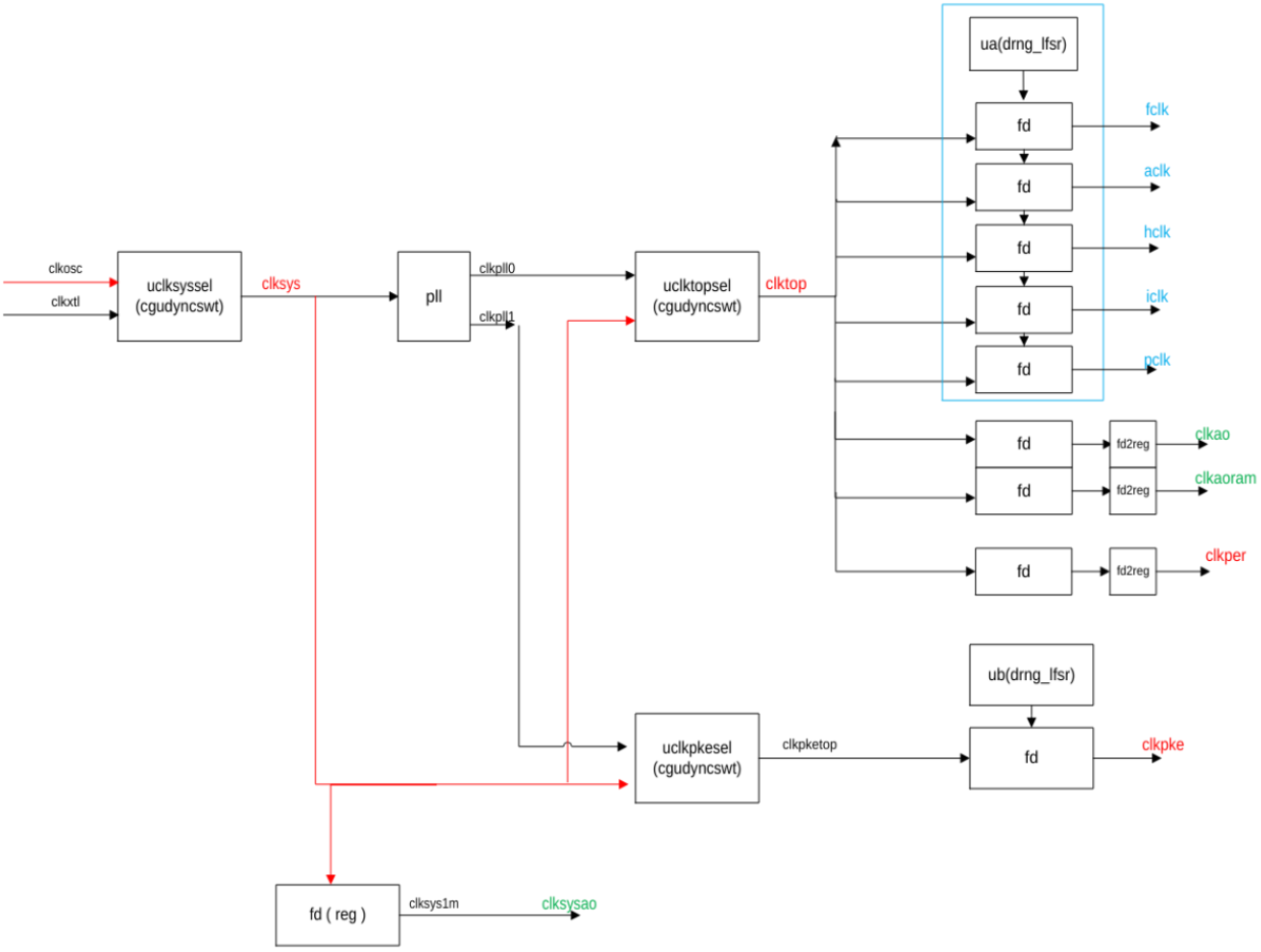
```
        # waiting for a while
HREAD    cgufsfreq0          # freq(MHz) of fclk / clkpke
HREAD    cgufsfreq1          # clkao / clkaoram
HREAD    cgufsfreq2          # osc / xtal
HREAD    cgufsfreq3          # pll0 / pll1
HREAD    cgufsvld              # for bit=0, meaning the measured clock is stopped.
```

Frequency meters need a while to count the clock cycles. Will not be accurate if the target clock is changing or the fs setting is just updated.

To setup pll

```
HWRITE   400400a0 0x143e8
HWRITE   400400a4 0x1000000
HWRITE   400400a8 0x1302
HWRITE   40040090 0x32          #setpll AR, to apply the settings to PLL
```

2.1.2 Clock initialization



The boot up status is:

- Clksys = clkosc
- Pll off
- All subgates are on
- Clktop = clksys
- fclk = aclk = hclk = iclk = pclk, fd is 0x7f, ½ clksys, about ~16MHz
- clkao, clkaoram, fd is 0xf, 1/16 clksys, ~2MHz
- Random clk off (ua)

To initial setup the clksys into:

- clksys = clkxtl

- pll on, pll0 = 500MHz
- clktop = clkpll0
- fclk = 500MHz, aclk = 250MHz, hclk = 125MHz, iclk = 62.5MHz, pclk =31.25MHz
- clkao, clkaoram ~8MHz

The software steps are:

- Check the status:
  - Read fs to check the status
- Change the clksys to xtal
  - HWRITE 40040030 0x01 #cgusel1, clksys=clkxtl
  - Read fs to check the last operation
- Enable the PLL
  - Setup PLLCR and PLLAR
  - Read fs to check clkpll0/1
- Change clk fd
  - HWRITE 40040010 0x1 #clktop/per=pll0
  - HWRITE 40040014 0x00001fff #fd fclk
  - HWRITE 40040018 0x00011f7f #fd aclk
  - HWRITE 4004001c 0x00033f3f #fd hclk
  - HWRITE 40040020 0x00073f1f #fd iclk
  - HWRITE 40040024 0x000f3f0f #fd pclk
  - HWRITE 40040028 0x003f3f07 #fd ao
  - HWRITE 40040038 0x3f07 #fd aoram
  - HWRITE 40040034 0x0f #fd pke
  - HWRITE 4004003c 0x07077fff #fd per
  - HWRITE 4004002c 0x32 #setcfg, to apply all above settings
  - Read fs to check the frequency
-
- Set random clk
  - HWRITE 40040000 0x1B1B # enable ua/ub, level = B (~70% clk)
  - Randomly reseed.

- Software random delay
- HWRITE 40040008 <randomseed>
- HWRITE 4004000C 0x5a #set reseed AR.

2.1.3 Clock frequency control and power control

To have different performance, we need to setup:

- Clk settings: pll, fd
- Voltage: 0.8V or 0.9V
  - Top speed 500MHz @0.8V
  - Top speed 800MHz @0.9V
- SRAM1 read cycle
  - SRAM1 is 2 cycle read @iclk=400MHz,0.9V
  - SRAM1 is 2 cycle read @iclk=250MHz,0.8V
  - SRAM1 is 1 cycle read @iclk<=200MHz,0.9V
  - SRAM1 is 1 cycle read @iclk<=125MHz,0.8V
- TCM fastmode settings
  - TCM set fastmode @fclk=800MHz, 0.9V
  - TCM set fastmode @fclk=500MHz, 0.8V
- ram tramming:
  - All sram macros are using different trimming when the voltage change
- Reram read cycle setting

The software steps are:

From 0.8V fclk 250MHz/aclk 125MHz

- Set the SRAM1 read cycle and iTCM fastmode before changing to top speed
  - HWRITE 40014010 0x4 # set SRAM1 read cycle = 1, set in lower freq

- HWRITE 40014004 0x1 # set iTCM in fastmode, set in lower freq

- Set fclk=500MHz, set aclk=250MHz

Continue to increase to voltage for 800MHz

- Set clktop = clksys
  - HWRITE 40040010 0x0 #clktop/per=clksys
  - Set fd
  - HWRITE 4004002c 0x32 #setcfg, to apply all above settings
- Set back the SRAM1 read cycle and iTCM fastmode after changing back to lower speed
  - HWRITE 40014010 0x0 # set SRAM1 read cycle = 0, set in lower freq
  - HWRITE 40014004 0x0 # set iTCM in fastmode = 0, set in lower freq
- Turn off PLL (optional)
- Change all sram trimming settings before changing to 0.9V
- Change voltage to 0.9V
  - Internal voltage changing by ipflow function
  - External volage changing depends on the PMIC settings
  - Check the voltage after changing
- Set clktop = clkxtl
- Turn on PLL, pll0 = 800MHz
- Set the SRAM1 read cycle and iTCM fastmode before changing to top speed
  - HWRITE 40014010 0x4 # set SRAM1 read cycle = 1, set in lower freq
  - HWRITE 40014004 0x1 # set iTCM in fastmode, set in lower freq
- Set clktop = clksys
  - HWRITE 40040010 0x0 #clktop/per=clksys
  - Set fd: fclk=ff as 800MHz, aclk=7f as 400MHz
  - HWRITE 4004002c 0x32 #setcfg, to apply all above settings
- Check all settings

2.2 Low power applications

To sleep CM7 only:

- Set CM7.SCR.deepsleep = 0
- WFI/WFE
- < enabled interrupt / event coming >
- Wakeup and jump into the handler program

Sleep with auto switching to fdlp setting, auto PLL off

- Set CM7.SCR.deepsleep = 1
- HWRITE 0x40040004 03 # cgulpcr=3 : fdlp and pll off
- HWRITE 0x40040010 1 # clktop/per = pll0, =clksys@deepsleep
- WFI/WFE
- < enabled interrupt / event coming >
- Wakeup and jump into the handler program, and fd and pll will be auto switched back

Deepsleep (cgulpcr.deepen = 1)

- Set CM7.SCR.deepsleep = 1
- HWRITE 0x40040004 07 # cgulpcr=7 : fdlp and pll off
- HWRITE 0x40040010 1 # clktop/per = pll0, =clksys@deepsleep
- Set ipc.ipc_lp to sleep all the analogs
- Set AO sysctrl to adjust the voltage (only for internal voltage) during sleepmode
- WFI/WFE
- < enabled async interrupt coming >
- Wakeup and jump into the handler program, and ip / fd / pll will be auto switched back

** To turn off XTAL, the system should keep osc enabled. Check the SDK driver for reference.

Powerdown mode, the whole Soc domain will be power off, AO domain will remain powered.

- HWRITE 40060000 0x7 # clk32k is from xtal, power isolation enable, icg enabled
- HWRITE 40060034 06664 #optional osc32k setting
- HWRITE 40040090 57 #ipflow set
- Write necessary flags or data to AORAM and AOBUREG
- Setup power (internal / external)
- HWRITE 40060044 5a #pd mode AR

When wakeup, the whole soc is restarted to bootup. A few things for boot to identify the sleep and wakeup:

- Read the AORAM / AOBUREG
- Read wakeup source flags from sfr_aofr (due to the low clock 32k and async signal could be various, the flag is just for additional ref, cant be recognized as the main evidence of the event)

3. Design Review Information

Topic of Review:
Date:

Related Documents:

Meeting Moderator:

Meeting Scribe:

Invitees	Attended (Y/N)	Invitees	Attended (Y/N)

Agenda/Issues discussed:

Outcome/Summary:

ID	Item	Owner	Date Due	Status	Closed?
1					
2					
3					

Table 3-1 Actions Items